Week 1: Project Proposal

Anh Huynh

Project Pitch

Game Title: Epic Brawlers

Description:
Epic Brawlers is a click-to-fight boss game built with Java Swing. The player faces one enemy boss at a time. Clicking **attack** inflicts damage; defeating a boss awards gold, which the player can spend on **weapon upgrades** to increase damage per click. Selecting to move onto the **Next Boss** spawns a tougher opponent with higher health and larger rewards.

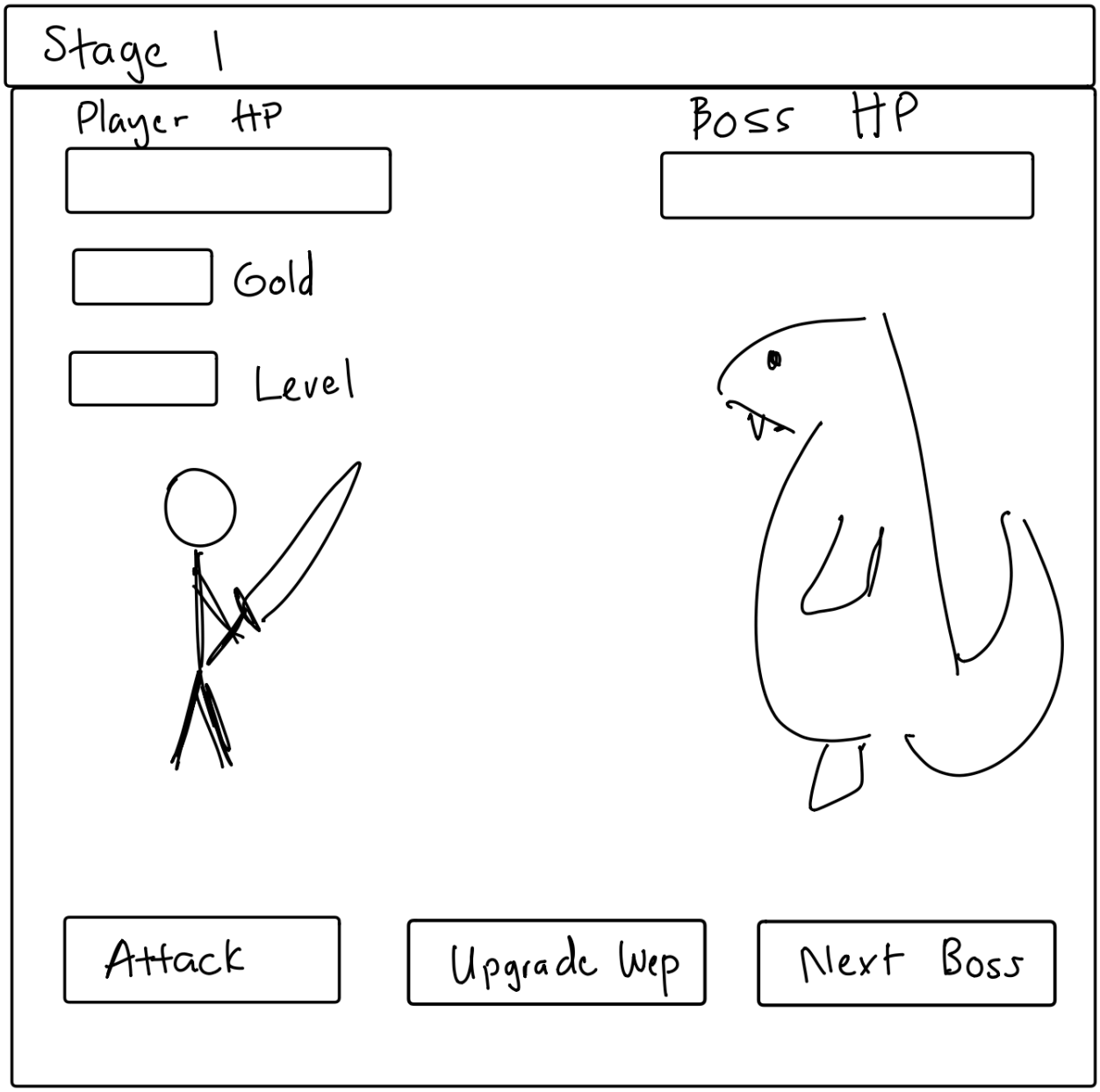
All gameplay happens through GUI components — buttons, labels, and a progress bar. There's no player movement; instead, progression focuses on stages, upgrades, and incremental strength.

Gameplay each round:
When the player clicks Attack, the player deals damage to the boss, and immediately after, the boss attacks back. This ensures that both sides lose HP each round. When the boss's HP reaches 0, the player earns gold and has the option to upgrade their weapon (upgrades unavailable if funds are insufficient) or advance to the next stage, where the next boss has higher HP, greater attack damage, and better rewards.

User actions:

- Clicks **Attack** —> Boss HP decreases. If HP <= 0, the boss dies, the player earns gold, and a new boss appears.
- Clicks **Upgrade Weapon** —> if enough gold, the player's weapon level increases, raising it's attack damage
- Clicks **Next Boss** —> Generates the next-tier boss.
- **Stage Progression** —> defeating a boss allows to advance to the next stage
- **Death/Reset** —> if HP <= 0, the player restarts at Stage 1, fully healed.

GUI Preview



CRC Cards

Player: tracks gold, level, damage, hp; handles upgrades and taking damage.
Collaborator(s): *Boss*

Boss: stores hp, maxHp, reward, attack, stage; handles taking damage
Collaborator(s): *Player*

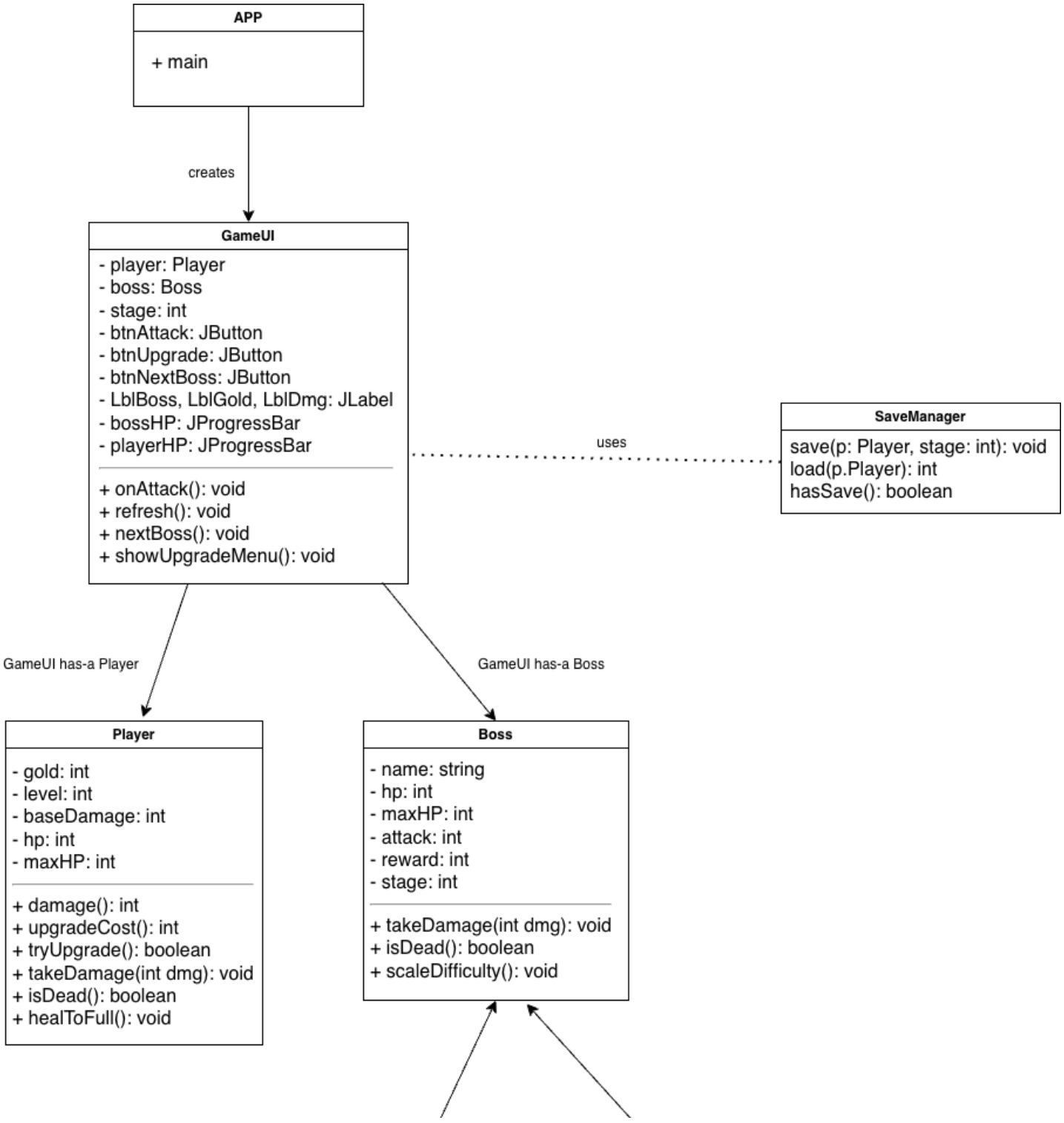
GameUI: manages GUI display and logic; responds to button clicks; updates HP bars and stage progression
Collaborator(s): *Player, Boss*

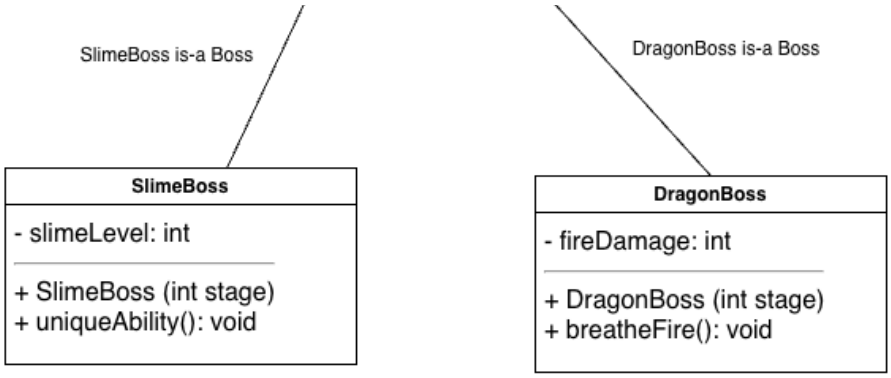
*__BaseWeapon:__ base class for all weapons; defines common fields (name, damage, level) and methods for attacking and upgrading

Collaborator(s): *Player, Boss*

*= possibly added in future

UML Diagram:





Object-Oriented Design

- Aggregation: The GameUI has-a Player and a Boss
- Encapsulation: Each class manages its own state and behavior (damage, HP, gold).
- Polymorphism: future versions include subclasses such as a SlimeBoss and DragonBoss to demonstrate inheritance
- Event-Driven Programming: each click triggers ActionListener events, updating the GUI and progressing the game round-by-round

Video Link: https://youtu.be/fAck9a8_tyI

Learning Outcomes

LO	Demonstrated in Epic Brawlers
LO1: Object Oriented Design Principles	Clear separation of classes; encapsulated Player/Boss classes; GUI/controller in GameUI class
LO2: · LO2: Construct programs utilizing single and multidimensional arrays (optional)	Optional
LO3: Objects & Aggregation	GameUI aggregates Player and Boss and coordinates their interactions
LO4: Inheritance & polymorphism	Planned boss hierarchy (e.g., BaseBoss —> SlimeBoss, DragonBoss) without changing controller code
LO6: GUI & Event-Driven Programming	Swing (JFrame, JButton, JLabel, JProgressBar) with ActionListener callbacks for attacks/upgrades
LO7: Exception handling	Handling for insufficient funds on upgrade; safe HP bounds; death/reset flow
LO8: Text File I/O	Save/load player progress (gold, level) to a simple text file

Planned Working Time

Week	Hours (outside class)
Week 1	Thurs 2-5 PM + Fri 6-9 PM
Week 2	Thurs 2-5 PM + Fri 6-9 PM
Week 3	Thurs 2-5 PM + Fri 6-9 PM / Sat 11 AM - 2 PM
Week 4	Thurs 2-5 PM + Fri 6-9 PM / Sat 11 AM - 2 PM
"..."	"..."

TO-DO Timeline Plan

Week	Goals
1	Write proposal; design CRC cards and UML; set up project files.
2	Implement Player and Boss classes; test damage logic and gold system.
3	Finalize model logic; create beta GUI layout. Implement rough codes for all other classes
4	Test and refine attack flow; prepare a simple non-functional GUI.
5	Test the GUI (view) and display live HP updates.
6	Connect event handling (controller) for attack and upgrade buttons.
7	Debug and polish stage progression and death/reset behavior.
8	Finalize all documentation and code; thorough testing

GitHub Repository

https://github.com/anhpls/epicbrawlers

Week 1 Additional Deliverables:

· *Deliverable 2 (optional, on the Canvas Project submission):* If you have started writing code, submit the code you have written so far, even if it is not complete and/or has compiler errors.

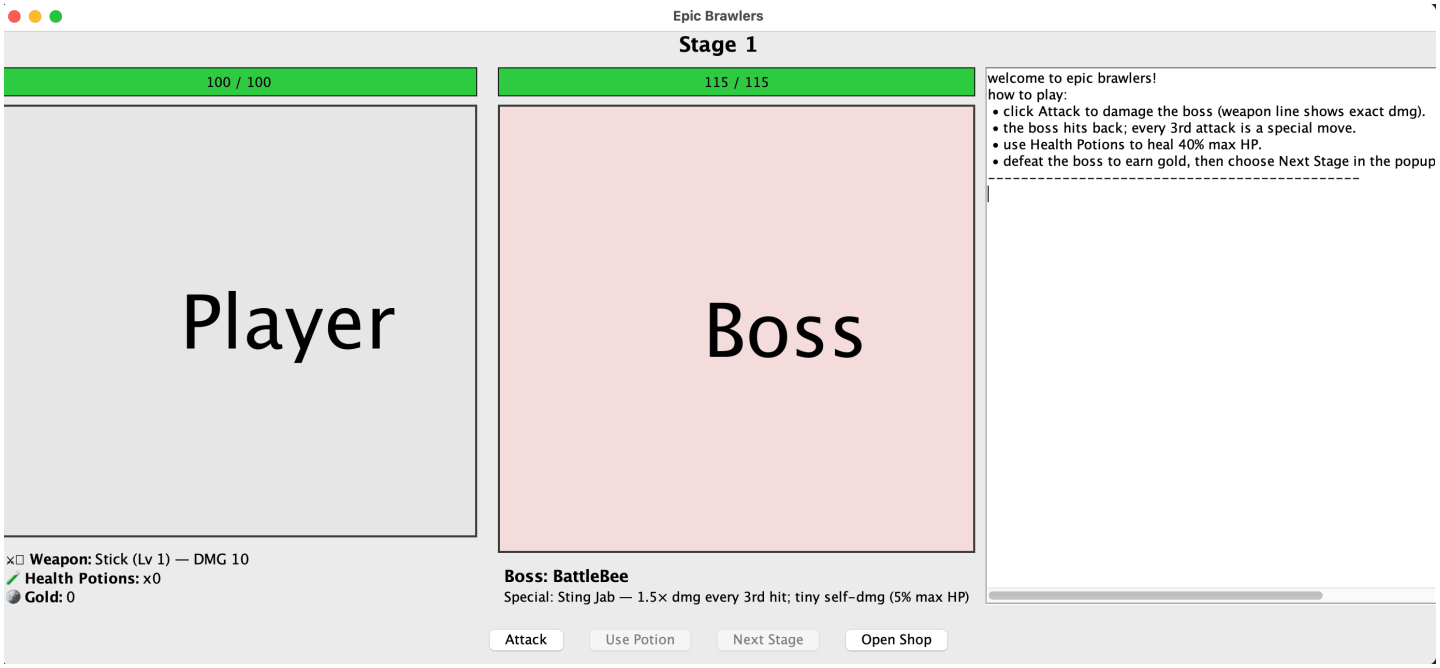
Week 2: Updates

Revise anything that is needed in the Week 1 section based on instructor feedback. Make sure your design is viable before working on your Week 2 goals from your project timeline.

Week 2 Deliverables

Deliverable 1 (on the Canvas Project submission): Submit the code you have written so far, even if it is not complete and/or has compiler errors.

Deliverable 2 (here): Update your Canvas Project page from Week 1. Please add to the page - do not delete any content from Week 1:



Journal Entry

This week, I focused on implementing the weapon classes and refining how damage scaling works across different weapon levels. Each weapon (Stick, Bow, Sword, and Magic Staff) now scales its damage using an exponential growth formula, which makes upgrades feel more impactful as the player progresses. I tweaked my original game design to make character progression more interesting. Instead of a straightforward "defeat boss → move to next stage" flow, I added the idea of upgrading weapons and revisiting older bosses** to grind for gold and build up strength through upgrades. This change adds more strategy to the gameplay and makes progression feel more rewarding.

Design Changes

The biggest design change this week was shifting the focus from a linear boss-fight system to a more upgrade-based loop. Players can now choose to replay stages to earn gold, purchase weapon upgrades, and boost their survivability before moving on to tougher bosses. This aligns better with idle and incremental RPG mechanics and gives players more freedom to decide how to progress.

Additionally, I improved the internal class design by introducing BaseWeapon and BaseBoss as parent classes. This structure uses inheritance and polymorphism to make it easier to define unique behavior for each subclass while sharing core logic like health, attack, and scaling.

Challenges Encountered

Balancing the damage output between the player and bosses was one of the main challenges. Early versions of the scaling system made weapons either too weak or too overpowered at certain levels. I also had to carefully adjust each boss's special attack logic to make sure battles stayed fair but challenging. (Still a work in progress). Another issue was managing the interaction between the player's stats and boss behavior in a way that kept the gameplay loop consistent.

What's Next

Next week, I plan to:

- Improve the **GUI** for the battle screen.
- Display **health bars, gold, potions, and weapons** visually for better clarity.

- Add early versions of the **shop system** so players can spend gold on upgrades.
- Begin connecting player and boss logic through an early controller class.

Updated Timeline

Week	Goals
1	Write proposal; design CRC cards and UML; set up project files.
2	Implement Player and Boss classes; test damage logic and gold system.
3	Finalize model logic; improve GUI layout; possibly add intro screens to give backstory to the game; implement rough codes for all other functionalities (mainly the shop + save/load progress)
4	(heavy focus on backend) Test and refine attack flow; heavy testing on gameplay logic + design and character progression
5	(focus on frontend) test the GUI for bugs + improve GUI interactivity + include better visuals
6	Connect event handling (controller) for attack and upgrade buttons.
7	lots of time debugging + improving gameplay logic before final touches.
8	Finalize all documentation and code; thorough testing

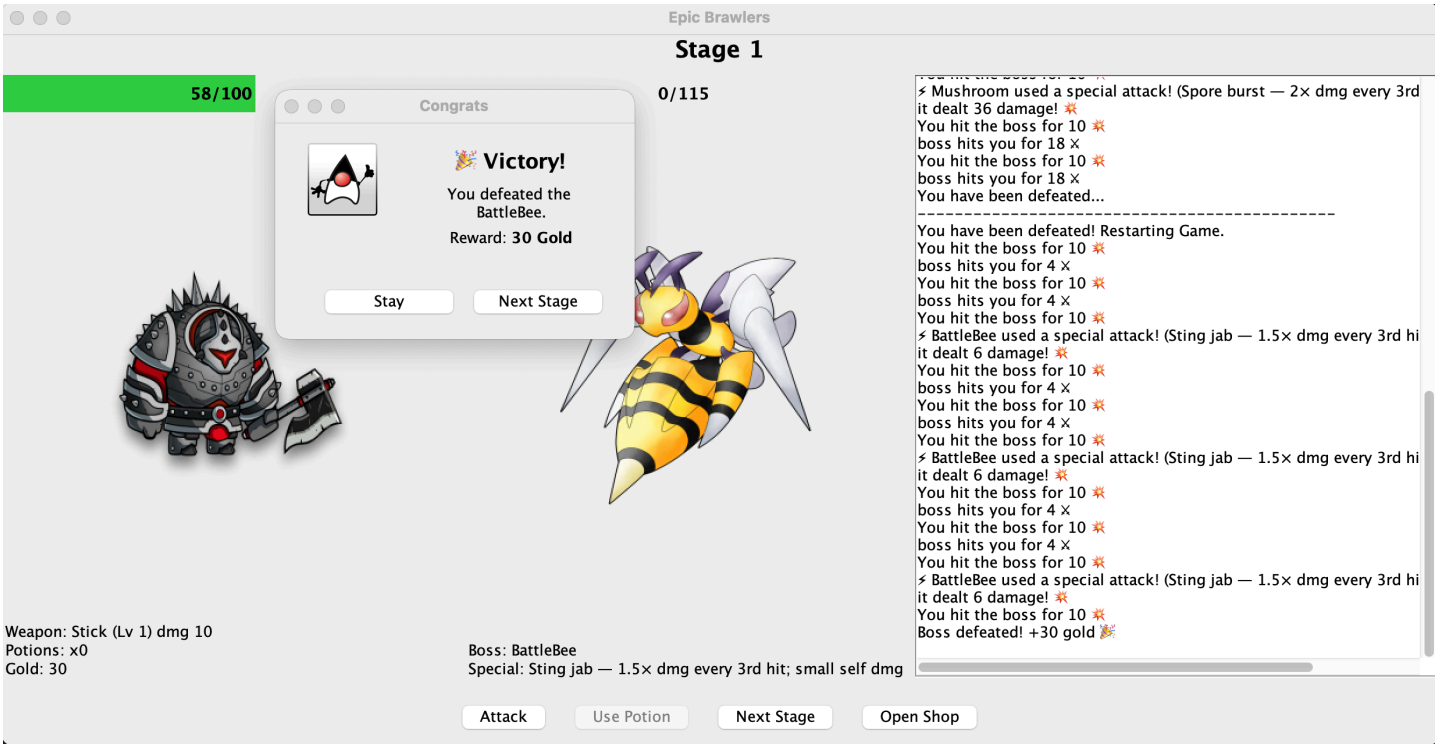
Week 3: Updates

Work on your Week 3 goals from your project timeline.

Week 3 Deliverables

Deliverable 1 (on the Canvas Project submission): Submit the code you have written so far, even if it is not complete and/or has compiler errors.

Deliverable 2 (here): Update your Canvas Project page from Week 1. Please add to the page - do not delete any content from Week 1 or Week 2:



Journal Entry

This week, I focused heavily on refining the Beta UI and fixing a long list of small but important issues that appeared once the battle screen became more visually complex. I spent time adjusting spacing, font sizes, and panel layouts to make the interface cleaner and easier to understand. I also added player and boss images, which required fixing scaling issues so the sprites would display correctly without stretching or overflowing their panels

Several interactive components needed logic fixes as well. For example, the “Use Potion” button now correctly disables itself when the player has no potions, and the “Next Stage” button only becomes enabled once the current boss is defeated. I also resolved problems with the health bars, which were initially not showing 0 HP correctly, or were displaying incorrect values after taking damage.

Week 3 was less about adding new gameplay mechanics and more about improving visual clarity and refining how the UI responds to game state changes, which helps the game feel more polished and functional.

Design Changes Made

- I replaced text labels for the player and bosses with actual images, making the game screen look more like a real RPG.
- I removed background panels and borders around the icons to create a cleaner, minimalist battle layout.
- I restructured parts of the UI logic so that boss images, stage labels, and button states update immediately whenever the player progresses or restarts.
- I added early support for a Re-Battle option where players can challenge previously defeated bosses to grind gold without advancing the stage.

These changes were made to make the game feel visually clearer and more interactive, and to better support the gameplay loop centered around progression and upgrading.

Challenges Encountered

This week's biggest challenges were UI-related:

- Image scaling caused stretched or oversized boss icons until I implemented a scaling helper method.
- The health bars didn't update properly at first; some values didn't display 0 HP or overflowed incorrectly.
- Button logic needed careful handling so players couldn't press actions at the wrong time (e.g., using potions when none were available).
- Some layout panels overlapped or misaligned, so I spent time adjusting layout managers, padding, and component sizes.

Debugging UI issues took more time than expected, but it helped solidify the underlying structure of the game.

What I Still Need to Do

To complete the project, I still need to:

- Implement the Shop System so players can spend gold on weapon upgrades and stats.
- Add Save/Load features or at least persistent player stats.
- Build more polished intro screens or story panels.
- Add more visual polish such as animations, effects, or sound (if time allows).
- Begin balancing boss stats, rewards, and difficulty curves.

Updated Timeline

Week	Goals
1	Write proposal; design CRC cards and UML; set up project files.
2	Implement Player and Boss classes; test damage logic and gold system.
3	Finalize model logic; improve GUI layout; possibly add intro screens to give backstory to the game; implement rough codes for all other functionalities
4	(heavy focus on GUI) Test and refine attack flow; heavy testing on GUI + include characters as well as enemies
5	(focus on frontend) test the GUI for bugs + improve GUI interactivity + include better visuals
6	Connect event handling (controller) for attack and upgrade buttons.
7	lots of time debugging + improving gameplay logic before final touches.
8	Finalize all documentation and code; thorough testing

Week 4: Updates

Work on your Week 4 goals from your project timeline.

Week 4 Deliverables

Deliverable 1 (on the Canvas Project submission): Submit the code you have written so far, even if it is not complete and/or has compiler errors.

Deliverable 2 (here): Update your Canvas Project page from Week 1. Please add to the page - do not delete any content from Weeks 1-3:

- Share a screenshot of the GUI.
- Add a journal-like entry about your experience writing code this week to your Canvas project page:
    - What design changes have you decided to make, and why did you make them?
    - What challenges have you encountered?
    - What do you still need to do to complete the project?
- Update your timeline goals, if needed.

Week 5: Updates

Work on your Week 5 goals from your project timeline.

Week 5 Deliverables

Deliverable 1 (on the Canvas Project submission): Submit the code you have written so far, even if it is not complete and/or has compiler errors.

Deliverable 2 (here): Update your Canvas Project page from Week 1. Please add to the page - do not delete any content from Weeks 1-4:

- Share a screenshot of the GUI.
- Add a journal-like entry about your experience writing code this week to your Canvas project page:
    - What design changes have you decided to make, and why did you make them?
    - What challenges have you encountered?
    - What do you still need to do to complete the project?
- Update your timeline goals, if needed.

Week 6: Updates

Work on your Week 6 goals from your project timeline.

Week 6 Deliverables

Deliverable 1 (on the Canvas Project submission): Submit the code you have written so far, even if it is not complete and/or has compiler errors.

Deliverable 2 (here): Update your Canvas Project page from Week 1. Please add to the page - do not delete any content from Weeks 1-5:

- Share a screenshot of the GUI.
- Add a journal-like entry about your experience writing code this week to your Canvas project page:
    - What design changes have you decided to make, and why did you make them?
    - What challenges have you encountered?
    - What do you still need to do to complete the project?
- Update your timeline goals, if needed.

Week 7: Updates

Work on your Week 4 goals from your project timeline.

Week 7 Deliverables

Deliverable 1 (on the Canvas Project submission): Submit the code you have written so far, even if it is not complete and/or has compiler errors.

Deliverable 2 (here): Update your Canvas Project page from Week 1. Please add to the page - do not delete any content from Weeks 1-6:

- Share a screenshot of the GUI.
- Add a journal-like entry about your experience writing code this week to your Canvas project page:
    - What design changes have you decided to make, and why did you make them?
    - What challenges have you encountered?
    - What do you still need to do to complete the project?
- Update your timeline goals, if needed.

Week 8: Project Wrap-up

Finish writing any other necessary code and debug (although hopefully you were debugging as you wrote the code!).

Week 8 Deliverables

Deliverable 1 (on the Canvas Project submission): Submit the code you have written so far, even if it is not complete and/or has compiler errors.

Deliverable 2 (here): Update your Canvas Project page from Week 1. Please add to the page - do not delete any content from Weeks 1-7:

- Share a screenshot of the text interaction with the user.
- Add a journal-like entry about your experience writing code this week to your Canvas project page:
    - What design changes did you make during the project, and why did you make them?
    - What challenges did you encounter?

- What could you do to expand on and improve your project?
- If you were to start the project from scratch, what would you do differently?

· Optionally, you may choose to share your code with your classmates by linking to your Eclipse project here.

Deliverable 3 (here): Share your video demonstration of your project and explanation of how and why the project utilized the concepts from the Learning Objectives; you only need to address the LOs for which you have not yet demonstrated Senior Developer proficiency. Aim for your video to be around 5 minutes, and do not exceed 10 minutes in length. Each partner must create their own video to demonstrate Senior Developer proficiency.

If you have not previously demonstrated Middle Developer proficiency for a LO, use your project code to make a video for that LO. Senior Developer proficiency will not be evaluated until Middle Developer proficiency is demonstrated.